Занятие 25.1. Пространства имен

Оглавление:

1. Конфликт имен
2. Что такое пространство имен?
3. Доступ к пространству имен через оператор разрешения области видимости (::)
4. Пространства имен с одинаковыми названиями
5. Псевдонимы и вложенные пространства имен

Конфликт имен

Конфликт имен возникает, когда два одинаковых идентификатора находятся в одной области видимости, и компилятор не может понять, какой из этих двух следует использовать в конкретной ситуации. Компилятор или линкер выдаст ошибку, так как у них недостаточно информации, чтобы решить эту неоднозначность. Как только программы увеличиваются в объемах, количество идентификаторов также увеличивается, следовательно, увеличивается и вероятность возникновения конфликтов имен.

Рассмотрим пример такого конфликта. boo.h и doo.h — это **заголовочные файлы** с функциями, которые выполняют разные вещи, но имеют одинаковые имена и **параметры**.

boo.h:

```
// Функция doOperation() выполняет операцию сложения своих параметров
int doOperation(int a, int b)
{
    return a + b;
}
```

doo.h:

```
// Функция doOperation() выполняет операцию вычитания своих параметров
int doOperation(int a, int b)
{
    return a - b;
}
```

main.cpp:

```
#include <iostream>
#include "boo.h"
#include "doo.h"

int main()
{
    std::cout << doOperation(5, 4); // какая версия doOperation() выполняется здесь?
    return 0;
}
```

Если boo.h и doo.h скомпилировать отдельно, то всё пройдет без инцидентов. Однако, соединив их в одной программе, мы подключим две разные функции, но с одинаковыми именами и параметрами, в одну область видимости (глобальную), а это, в свою очередь, приведет к конфликту имен. В результате, компилятор выдаст ошибку. Для решения подобных проблем и добавили в язык C++ такую концепцию, как пространства имен.

Что такое пространство имен?

Пространство имен определяет область кода, в которой гарантируется уникальность всех идентификаторов. По умолчанию, **глобальные переменные** и обычные функции определены в **глобальном пространстве имен.** Например:

```
int g_x = 4;

int boo(int z)
{
    return -z;
}
```

Глобальная переменная `g_x` и функция boo() определены в глобальном пространстве имен.

В примере, приведенном выше, при подключении файлов boo.h и doo.h обе версии doOperation() были включены в глобальное пространство имен, из-за чего, собственно, и произошел конфликт имен.

Чтобы избежать подобных ситуаций, когда два независимых объекта имеют идентификаторы, которые могут конфликтовать друг с другом при совместном использовании, язык C++ позволяет объявлять собственные пространства имен через **ключевое слово namespace**. Всё, что объявлено внутри пользовательского пространства имен, — принадлежит только этому пространству имен (а не глобальному).

Перепишем заголовочные файлы из вышеприведенного примера, но уже с использованием namespace:

boo.h:

```
namespace Boo
{
    // Эта версия doOperation() принадлежит пространству имен Boo
    int doOperation(int a, int b)
    {
        return a + b;
    }
}
```

doo.h:

```
namespace Doo
{
    // Эта версия doOperation() принадлежит пространству имен Doo
    int doOperation(int a, int b)
    {
        return a - b;
    }
}
```

Теперь doOperation() из файла boo.h находится в пространстве имен `Boo`, а doOperation() из файла doo.h — в пространстве имен `Doo`. Посмотрим, что произойдет при перекомпиляции main.cpp:

```
int main()
{
    std::cout << doOperation(5, 4); // какая версия doOperation() здесь выполняется?
    return 0;
}
```

Результатом будет еще одна ошибка:

```
C:\VCProjects\Test.cpp(15) : error C2065: 'doOperation' : undeclared identifier
```

Случилось так, что когда мы попытались вызвать функцию doOperation(), компилятор заглянул в глобальное пространство имен в поисках определения doOperation(). Однако, поскольку ни одна из наших версий doOperation() не находится в глобальном пространстве имен, компилятор не смог найти определение doOperation() вообще!

Существует два разных способа сообщить компилятору, какую версию doOperation() следует использовать: через оператор разрешения области видимости или с помощью using-стейтментов (о них мы поговорим на следующем уроке).

Доступ к пространству имен через оператор разрешения области видимости (::)

Первый способ указать компилятору искать идентификатор в определенном пространстве имен — это использовать название необходимого пространства имен вместе с оператором разрешения области видимости (`::`) и требуемым идентификатором.

Например, сообщим компилятору использовать версию doOperation() из пространства имен `Boo`:

```
int main(void)
{
    std::cout << Boo::doOperation(5, 4);
    return 0;
}
```

Результат:

```
9
```

Если бы мы захотели использовать версию doOperation() из пространства имен `Doo`:

```
int main(void)
{
    std::cout << Doo::doOperation(5, 4);
    return 0;
}
```

Результат:

```
1
```

Оператор разрешения области видимости хорош, так как позволяет выбрать конкретное пространство имен. Мы даже можем сделать следующее:

```
int main(void)
{
    std::cout << Boo::doOperation(5, 4) << '\n';
    std::cout << Doo::doOperation(5, 4) << '\n';
    return 0;
}
```

Результат:

```
9
1
```

Также этот оператор можно использовать без какого-либо префикса (например, `::doOperation`). В таком случае мы ссылаемся на глобальное пространство имен.

Пространства имен с одинаковыми названиями

Допускается объявление пространств имен в нескольких местах (либо в нескольких файлах, либо в нескольких местах внутри одного файла). Всё, что находится внутри одного блока имен, считается частью только этого блока.

add.h:

```
namespace DoMath
{
    // Функция add() является частью пространства имен DoMath
    int add(int x, int y)
    {
        return x + y;
    }
}
```

subtract.h:

```
namespace DoMath
{
    // Функция subtract() является частью пространства имен DoMath
    int subtract(int x, int y)
    {
        return x - y;
    }
}
```

main.cpp:

```
#include "add.h" // импортируем DoMath::add()
#include "subtract.h" // импортируем DoMath::subtract()

int main(void)
{
    std::cout << DoMath::add(5, 4) << '\n';
    std::cout << DoMath::subtract(5, 4) << '\n';

    return 0;
}
```

Всё работает, как нужно.

Стандартная библиотека C++ широко использует эту особенность, поскольку все заголовочные файлы, которые находятся в ней, реализуют свой функционал внутри пространства имен std.

Псевдонимы и вложенные пространства имен

Одни пространства имен могут быть вложены в другие пространства имен. Например:

```
#include <iostream>

namespace Boo
{
    namespace Doo
    {
        const int g_x = 7;
    }
}

int main()
{
    std::cout << Boo::Doo::g_x;
    return 0;
}
```

Обратите внимание, поскольку `Doo` находится внутри `Boo`, то доступ к `g_x` осуществляется через `Boo::Doo::g_x`.

Так как это не всегда удобно и эффективно, то C++ позволяет создавать псевдонимы для пространств имен:

```
#include <iostream>

namespace Boo
{
    namespace Doo
    {
        const int g_x = 7;
    }
}

namespace Foo = Boo::Doo; // Foo теперь считается как Boo::Doo

int main()
{
    std::cout << Foo::g_x; // это, на самом деле, Boo::Doo::g_x
    return 0;
}
```

Стоит отметить, что пространства имен в C++ не были разработаны, как способ реализации информационной иерархии — они были разработаны в качестве механизма предотвращения возникновения конфликтов имен. Как доказательство этому, вся Стандартная библиотека шаблонов находится в единственном пространстве имен `std::`.

Вложенность пространств имен не рекомендуется использовать, так как при неумелом использовании увеличивается вероятность возникновения ошибок и дополнительно усложняется логика программы.